

Interfaces Conversacionales en Physical AI — Análisis completo de la Sesión 18

Fuente base: *Agentes IA — Physical AI* (SES18_M6_InterfacesFisicas.pdf , 17 diapositivas) — Módulo 6 (Interacción con el Mundo Físico), Programa en Diseño e Implementación de Agentes IA, UTEC Posgrado. Dictada por MEng. Boris Alzamora.

Nota técnica: el PDF tiene poca capa de texto extraíble en varias diapositivas clave (6, 7, 8, 14) — son diagramas embebidos como imagen. Este documento se generó **renderizando las 17 páginas e interpretándolas visualmente**, complementado con el código completo del laboratorio adjunto (agents26_m6s18-main/), que implementa exactamente los patrones que la teoría describe.

Hallazgo clave de esta sesión: las diapositivas 6, 7 y 8 reproducen, con el mismo vocabulario y el mismo estilo de diagrama, la guía oficial **"Voice Agents"** de OpenAI (platform.openai.com / openai.github.io/openai-agents-js), que define exactamente dos arquitecturas — **Chained** (STT→LLM→TTS) y **Speech-to-Speech** (modelo multimodal nativo) — más un patrón de *multi-agente por voz* con *handoff* a agentes especializados. Esta sesión es también donde el programa pasa de la teoría de multimodalidad (Sesión 17) a su implementación **en tiempo real y bidireccional** — el requisito de latencia que separa "un agente que ve/oye" de "un agente con el que se puede conversar".

1. Objetivos y Agenda

Objetivos declarados:

- 1. Entender las interfaces conversacionales en *Physical AI*.
- 2. Entender la extensibilidad del entorno y del *context* por *Physical AI*.

Agenda (dos partes, diapositivas 5 y 13):

#	Tema
1	Interfaces Conversacionales en Physical AI
2	Voice and video
3	Stream
4	WebSockets and RPC
5	SaaS products

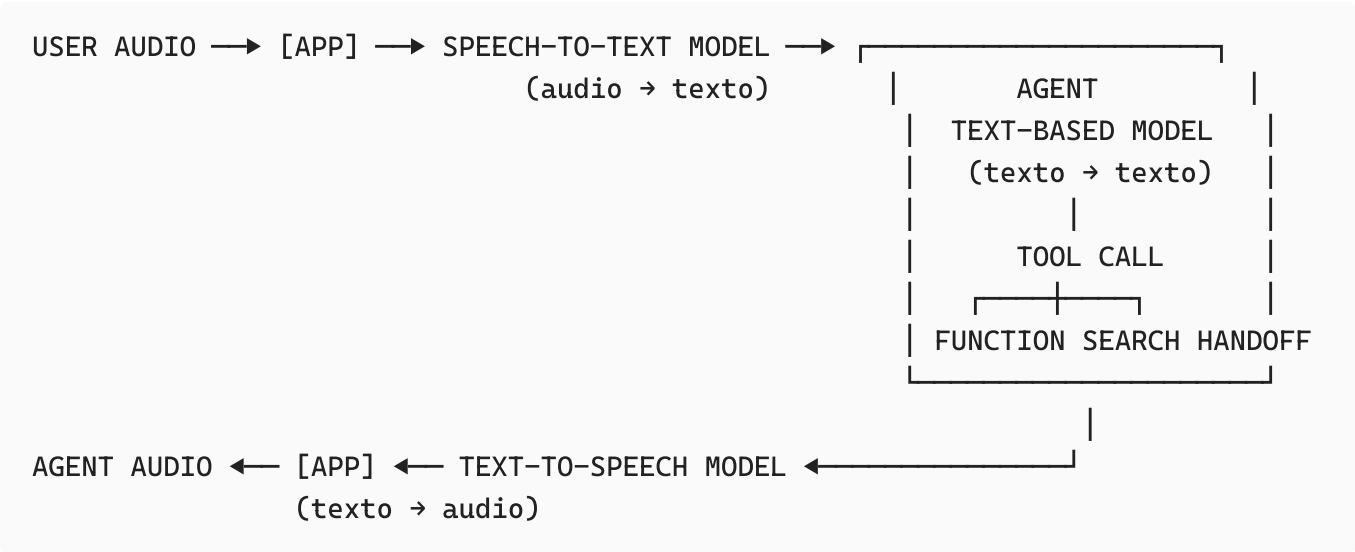
#	Tema
6	ElevenLabs

Esta sesión es breve (17 diapositivas frente a las 49 de la Sesión 17) pero muy densa en contenido técnico de *networking* — es la pieza que le falta al Módulo 6 antes de robótica: **cómo conectar, en tiempo real y con baja latencia, la voz de un usuario humano con un agente de IA** (y viceversa), el requisito de infraestructura para cualquier interfaz física conversacional (altavoces inteligentes, robots, *call centers* con IA, gafas AR).

2. Las 3 arquitecturas de agentes de voz (diapositivas 6-8)

Estas tres diapositivas reproducen, con el mismo estilo de diagrama, la guía oficial de OpenAI "**Voice Agents**", que documenta dos arquitecturas fundamentales para construir agentes conversacionales por voz, más un patrón avanzado de orquestación multiagente.

2.1 Arquitectura *Chained* (encadenada) — diapositiva 6



El audio del usuario se convierte a texto (STT), un **modelo basado en texto** (el LLM habitual, con *tool calling*: FUNCTION , SEARCH , HANDOFF) procesa la conversación como si fuera texto puro, y la respuesta se convierte de vuelta a audio (TTS) antes de devolverla. **Es la misma arquitectura, con las mismas tres piezas (STT, LLM+tools, TTS), que implementa el laboratorio de esta sesión (§5).**

2.2 Arquitectura *Speech-to-Speech* — diapositiva 7



Un **agente de primera línea** ("frontline") conversa por voz en tiempo real con el usuario usando la *Realtime API*, y cuando la conversación requiere una especialidad concreta, hace *tool calls* hacia agentes de **texto** especializados — cada uno corriendo un modelo distinto elegido según el costo/capacidad que exige su tarea: `o3` (razonamiento) para reembolsos, `o4-mini` (más barato) para cancelación de pedidos, `GPT-4.1` para preguntas de producto. **Es el mismo patrón "Supervisor (as tools)" / Orchestrator-Workers ya documentado en la Sesión 15** (`Sesion15_LangGraph_MultiAgent_ANALISIS_COMPLETO.md` §6.1), aplicado aquí a un canal de voz: el agente de voz nunca deja de hablar con el usuario — delega el *razonamiento* de una subtarea a un agente de texto, pero la interfaz conversacional permanece continua.

3. WebSocket (diapositiva 9)

"WebSocket es un protocolo de comunicación que permite una conexión bidireccional y persistente entre cliente y servidor. A diferencia de HTTP, WebSocket mantiene el canal abierto, lo que permite enviar y recibir datos en tiempo real sin necesidad de múltiples solicitudes."

Características clave según el material:

Característica	Qué aporta
Comunicación en tiempo real	Ideal para chats, juegos en línea, <i>trading</i> , apps colaborativas
Bajo consumo de recursos	Reduce la sobrecarga de HTTP al evitar múltiples conexiones
Full-duplex	Cliente y servidor pueden enviar mensajes simultáneamente
Persistencia	La conexión se mantiene abierta, mejorando la eficiencia

Por qué esto importa para un agente de voz: un agente conversacional no puede esperar a que termine una petición HTTP para "escuchar" la siguiente frase — necesita un canal **abierto en ambas direcciones a la vez** (el usuario puede seguir hablando mientras el agente aún está respondiendo). WebSocket es la base de transporte típica tanto de la *Realtime API* de OpenAI como de la mayoría de SDKs de voz en tiempo real (incluido `fastrtc` , usado en el laboratorio, §5).

4. RTC — Real-Time Communication (diapositiva 10)

"RTC se refiere a tecnologías que permiten la transmisión de audio, video y datos en tiempo real entre dispositivos a través de internet. Es la base de aplicaciones como videollamadas, chats en vivo y colaboración remota."

Tecnologías clave citadas:

Tecnología	Rol
WebRTC	API de código abierto que permite comunicación directa <i>peer-to-peer</i> entre navegadores, sin <i>plugins</i>
STUN / TURN / ICE	Protocolos que ayudan a establecer y mantener la conexión entre pares (atravesando NAT/ <i>firewalls</i>)
SRTP	Protocolo seguro para la transmisión de medios (audio/video) en tiempo real

Cómo funciona (según el material):

1. Se establece una conexión *peer-to-peer* entre dos dispositivos.
2. Se negocia el intercambio de medios (audio/video) y datos.
3. Se transmite el contenido con baja latencia y alta eficiencia.

Aplicaciones comunes: videollamadas (Zoom, Google Meet), chats en vivo y atención al cliente, juegos multijugador, colaboración en tiempo real (documentos, pizarras compartidas).

La diferencia práctica entre WebSocket y WebRTC (no explicitada en el material, pero necesaria para entender por qué existen ambos): WebSocket transporta **datos genéricos** (texto, JSON, *bytes*) por un canal cliente-servidor; WebRTC está optimizado específicamente para **audio/video de baja latencia**, con compresión, corrección de errores y negociación *peer-to-peer* incorporadas — por eso las librerías de agentes de voz (como `fastrtc`, §5) se construyen sobre WebRTC y no sobre WebSocket puro cuando el medio es audio en vivo.

5. El laboratorio — de la teoría al código (`agents26_m6s18-main/`)

El repositorio adjunto contiene **dos ejercicios progresivos** que, juntos, cubren exactamente las dos mitades de esta sesión: primero el *streaming* de texto de un agente (la base de cualquier interfaz conversacional), y después un agente de **voz completo en tiempo real** usando `fastrtc`.

5.1 Streaming de agentes con LangChain ≥ 1.0

(agent_streaming.py , agent_streaming_avanzado.py)

Ambos scripts usan `create_agent` (la API vigente en LangChain ≥ 1.0, que reemplaza al `initialize_agent` ya deprecado) y muestran **las dos formas de hacer streaming** documentadas en el `README.md` del laboratorio:

Script	Mecanismo	Qué expone
agent_streaming.py	<code>agent.stream(..., stream_mode="messages")</code> (síncrono)	Tokens del LLM a medida que se generan, entregados como tuplas (<code>chunk</code> , <code>metadata</code>) — la forma más simple
agent_streaming_avanzado.py	<code>agent.astream_events(..., version="v2")</code> (asíncrono)	Eventos granulares: <code>on_chat_model_stream</code> (token nuevo), <code>on_chat_model_start/end</code> , <code>on_tool_start/end</code> , <code>on_chain_start/end</code> — permite mostrar en vivo qué herramienta se está ejecutando y con qué argumentos, no solo el texto final

Un detalle de ingeniería que documenta el propio código: el contenido de un `AIMessageChunk` puede llegar como un `string` plano o como una lista de bloques `[{"type": "text", "text": "..."}]` según el proveedor — ambos scripts incluyen una función `extraer_texto()` que normaliza los dos formatos antes de imprimir. Es un recordatorio útil de que **el formato exacto de un chunk de streaming no es un estándar universal entre proveedores**, incluso dentro del mismo framework.

Por qué el streaming de texto es prerequisite de un agente de voz: en la arquitectura *Chained* (§2.1), el TTS no puede esperar a que el LLM termine toda su respuesta para empezar a hablar — necesita poder sintetizar audio **frase por frase, a medida que el LLM va generando tokens**. Sin *streaming* a nivel de texto, la latencia percibida de un agente de voz encadenado sería inaceptable (el usuario esperaría en silencio varios segundos).

5.2 El agente de voz completo (agent.py + main.py) — implementación real de la arquitectura Chained

Este es el ejercicio central del laboratorio, y reproduce **literalmente** el diagrama de la diapositiva 6 (§2.1) con librerías reales:

```
# agent.py – el "AGENT / TEXT-BASED MODEL" del diagrama
class agent_openai:
    self.llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.1)
    self.sysprompt = "Eres un agente de WebRTC... respuestas concisas y cortas, sin emojis."
    self.tools = [self.get_systemtime] # FUNCTION del diagrama
```

```
# main.py – el pipeline STT → Agent → TTS completo
from fastrtc import ReplyOnPause, Stream, get_tts_model
from fastrtc_whisper_cpp import get_stt_model

stt_model = get_stt_model() # moonshine/base → "SPEECH-TO-TEXT MODEL"
tts_model = get_tts_model() # kokoro → "TEXT-TO-SPEECH MODEL"

def echo(audio):
    transcript = stt_model.stt(audio) # audio → texto
    response_text = agent_openai().llm.invoke(input=transcript) # texto → texto (AGENT)
    for audio_chunk in tts_model.stream_tts_sync(response_text.content):
        yield audio_chunk # texto → audio, en streaming

stream = Stream(ReplyOnPause(echo), modality="audio", mode="send-receive")
stream.ui.launch()
```

Pieza del diagrama (§2.1)	Implementación real en el código
USER AUDIO / AGENT AUDIO	Capturados/reproducidos por la UI que lanza <code>fastrtc (stream.ui.launch())</code>
[APP] (las burbujas de transporte)	La librería <code>fastrtc</code> — provee <code>Stream</code> (el canal WebRTC full-duplex, <code>mode="send-receive"</code>) y <code>ReplyOnPause</code> (detecta cuándo el usuario dejó de hablar para disparar la respuesta, evitando que el agente interrumpa a mitad de frase)
SPEECH-TO-TEXT MODEL	<code>fastrtc_whisper_cpp</code> con el modelo moonshine/base — una variante ligera de Whisper pensada para ejecutarse localmente con baja latencia
TEXT-BASED MODEL (Agent)	<code>ChatOpenAI(model="gpt-4o-mini")</code> , con <i>system prompt</i> explícitamente diseñado para voz: <i>"respuestas concisas y cortas, sin uso de emojis"</i> —

Pieza del diagrama (§2.1)	Implementación real en el código
	un detalle de diseño importante: un emoji o una respuesta larga no tienen sentido cuando el destino final es un sintetizador de voz
TEXT-T0-SPEECH MODEL	kokoro — modelo TTS de código abierto, invocado con <code>stream_tts_sync()</code> , que entrega el audio en <i>chunks</i> a medida que se genera (streaming real, no espera al texto completo)

Esto conecta directamente con la Sesión 17

(Sesion17_Multimodalidad_ANALISIS_COMPLETO.md §4): es, otra vez, el **Camino 1 — LLM + Tools**, pero aplicado ahora al par de modalidades audio↔texto con *streaming* en ambos sentidos, en vez de al par texto↔imagen sin streaming del laboratorio de siniestros. El mismo principio de diseño ("orquestrar modelos ya entrenados, sin *fine-tuning*, alrededor de un LLM de texto") se sostiene incluso cuando la exigencia de tiempo real sube considerablemente.

5.3 requirements.txt — nota de versiones

El archivo incluye un comentario explícito y fechado ("*Versiones verificadas y probadas end-to-end el 2026-08-13*") fijando `openai<3.0.0` porque, a esa fecha, `langchain-openai>=1.4.3` todavía no soportaba el `major openai==3.0.0` por *breaking changes* — un recordatorio práctico de que en un ecosistema de dependencias que evoluciona tan rápido (LangChain ≥1.0, OpenAI SDK en v2.x/v3.x), fijar rangos de versión probados explícitamente es una práctica necesaria, no opcional, para que un laboratorio siga siendo reproducible meses después.

6. SaaS products — ElevenLabs y el ecosistema de voz (diapositivas 13-15)

La segunda mitad de la sesión se dedica a plataformas SaaS especializadas en voz, en vez de construir cada pieza (STT/TTS) desde cero como en el laboratorio:

- **ElevenLabs** — la plataforma líder en síntesis y clonación de voz (ya había aparecido como logo en el mosaico de "mercado global" de la Sesión 17).
- **Twilio** — infraestructura de telefonía/mensajería programable; en este contexto, el puente típico para que un agente de voz pueda **recibir y hacer llamadas telefónicas reales** (no solo audio dentro de una app web).
- **Temas de exploración señalados: STT and TTS** (los mismos dos componentes ya cubiertos en el laboratorio, pero ahora como servicio administrado en vez de modelos

locales como moonshine/kokoro) y **Speech Agents** (agentes de voz llave en mano que ofrece ElevenLabs, análogos en concepto al `agent.py / main.py` del laboratorio pero sin necesidad de orquestar cada pieza manualmente).

La disyuntiva que plantea implícitamente esta sección, frente al laboratorio de §5: construir el *pipeline* de voz con librerías abiertas (`fastrtc` + `moonshine` + `kokoro` , todo *self-hosted*) da control total y costo marginal bajo, pero exige integrar y mantener cada pieza; usar una plataforma SaaS (ElevenLabs, Twilio) da *time-to-market* más rápido y voces de mayor calidad percibida, a cambio de costo por uso y menor control sobre la infraestructura. Es la misma disyuntiva "construir vs. comprar" que aparece en cualquier decisión de arquitectura de agentes.

7. Laboratorios y tarea de la sesión

Actividad	Instrucción
Lab — Explorando RTC	Explorar Stream , fastRTC y WebSockets — el mismo código de <code>agent.py / main.py</code> (§5.2).
Lab — Aterrizando a proyectos	Reflexionar: ¿podríamos integrar alguna interfaz de comunicación física en mi proyecto? <i>Check</i> : usar draw.io u otra herramienta para plasmarlo.
Lab — Explorando ElevenLabs	STT y TTS, y Speech Agents como plataforma SaaS.
Tarea — Physical AI Agent Interfaces	Tipo: grupal . Hacer una prueba de <i>Physical Agent Interfaces</i> con cualquier plataforma SaaS considerada para el proyecto del equipo, y documentar los hallazgos hacia dicho proyecto. Fecha límite: 12/11.

Nota de contexto: en esta misma carpeta ya existe un documento de diseño propio — `Physical_AI_Agent_Monitoreo_Gatos_Tapo_YOLO_MCP.md` (cámara Tapo C220 + RTSP + YOLO + LangChain/LangGraph + MCP, para monitoreo de gatos) — que encaja como candidato directo para esta tarea grupal: aunque su interfaz principal es visión (RTSP/YOLO) y no voz, el documento ya identifica explícitamente el patrón *Physical AI Agent Interfaces* como el marco de evaluación del proyecto. Si el equipo decide sumar una interfaz de voz (p. ej. para consultar el estado de los gatos hablando con el agente, usando el mismo patrón *Chained* de §2.1/§5.2), esta sesión provee directamente las piezas (STT/TTS locales vía `fastrtc` , o ElevenLabs/Twilio como alternativa SaaS).

8. Síntesis — lo que hay que llevarse de esta sesión

1. **Existen exactamente dos arquitecturas para un agente de voz** (guía oficial de OpenAI, §2): *Chained* (STT→LLM→TTS, predecible, fácil de depurar, la recomendada para empezar) y *Speech-to-Speech* (un modelo multimodal audio-a-audio, menor latencia y más matices de voz, más difícil de depurar). El laboratorio de esta sesión implementa la primera.
 2. **Un agente de voz puede escalar a multiagente sin dejar de ser conversacional** (§2.3): un agente de primera línea mantiene la conversación por voz y delega, vía *tool calls*, a agentes de texto especializados con modelos distintos según el costo que justifica cada subtarea — el mismo patrón Orchestrator/Supervisor-as-tools de la Sesión 15, aplicado a voz.
 3. **WebSocket y WebRTC no son intercambiables** (§3-4): WebSocket es el transporte genérico bidireccional para datos; WebRTC es la capa especializada (con STUN/TURN/ICE/SRTP) para audio/video de baja latencia — por eso las librerías de agentes de voz como *fastrtc* se construyen sobre WebRTC.
 4. **El *streaming* de texto es prerequisite de un buen agente de voz** (§5.1): sin poder emitir tokens progresivamente, ni el TTS puede empezar a hablar antes de que el LLM termine, y la latencia percibida se vuelve inaceptable.
 5. **El código del laboratorio (§5.2) es una instancia directa y verificable del diagrama teórico** (§2.1) — moonshine (STT) + gpt-4o-mini (agente de texto con *tools*) + kokoro (TTS), orquestados por *fastrtc*. Confirma, con evidencia de código, que el material no es solo teoría de diapositiva sino un patrón reproducible con herramientas de código abierto sin depender de un SaaS de voz.
 6. **La decisión SaaS vs. *self-hosted* para voz (§6) es una disyuntiva de "construir vs. comprar"** más, no una elección técnica superior/inferior — ElevenLabs/Twilio dan velocidad y calidad percibida; *fastrtc* + moonshine + kokoro dan control y costo marginal bajo.
-

9. Checklist práctico — diseñando una interfaz de voz para un agente

- ☐ ¿La latencia y los matices de voz (tono, emoción) son críticos para el caso de uso, o basta con una transcripción textual intermedia? → Si basta, usar *Chained* (más fácil de depurar); si no, considerar *Speech-to-Speech*.
- ☐ ¿El agente necesita delegar subtarear a modelos distintos según costo/capacidad (p. ej. un modelo caro solo para razonamiento de reembolsos)? → Considerar el patrón *frontline agent + tool calls a agentes especializados* (§2.3).

- ☐ ¿El transporte de audio necesita ser *peer-to-peer* de baja latencia (llamada en vivo) o basta un canal de datos bidireccional genérico? → Audio real en vivo → WebRTC (o una librería que lo envuelva, como `fastrtc`); datos/eventos → WebSocket puro.
 - ☐ ¿El LLM de texto del pipeline emite su respuesta con *streaming* token a token? Sin esto, el TTS no puede empezar a sintetizar antes de que termine toda la respuesta.
 - ☐ ¿El *system prompt* del agente está adaptado a que su salida se convertirá en voz (respuestas cortas, sin *markdown*, sin emojis, sin URLs largas)?
 - ☐ ¿Construir el *pipeline* de voz con librerías abiertas (control total, costo marginal bajo, más mantenimiento) o con una plataforma SaaS (ElevenLabs, Twilio; más rápido de lanzar, menos control)?
 - ☐ Si el proyecto necesita telefonía real (recibir/hacer llamadas, no solo audio en una app web): ¿está contemplado un proveedor de telefonía programable (Twilio) además del STT/TTS?
 - ☐ ¿Las versiones de las librerías del *pipeline* (LangChain, OpenAI SDK, `fastrtc`) están fijadas y probadas end-to-end, dado lo rápido que rompen compatibilidad entre sí?
-

10. Referencias

Del material original:

- Diagramas propios del curso (diapositivas 6, 7, 8) — arquitecturas *Chained* y *Speech-to-Speech*, y el patrón multiagente con *Realtime API*.
- Definiciones de WebSocket y RTC (diapositivas 9-10).
- Logos/referencias de mercado: ElevenLabs, Twilio.

Investigación complementaria (añadida en este documento):

- OpenAI — guía oficial ["Voice Agents"](https://openai.github.io/openai-agents-js/guides/voice-agents) (y su equivalente en `openai.github.io/openai-agents-js/guides/voice-agents`): fuente directa de las arquitecturas *Chained* y *Speech-to-Speech* de las diapositivas 6-7, incluyendo la recomendación explícita de empezar por *Chained* por ser más predecible y fácil de depurar.
- Verificación de que el patrón de la diapositiva 8 (agente de primera línea con *tool calls* a agentes de texto especializados por modelo) corresponde al mismo patrón de orquestación multiagente (*Supervisor as tools / Orchestrator-Workers*) documentado con las topologías oficiales de LangGraph en la Sesión 15.
- Análisis propio del código del laboratorio (`agents26_m6s18-main/`, §5): trazabilidad completa de cómo `agent.py` + `main.py` implementan, pieza por pieza, el diagrama teórico de la arquitectura *Chained* usando `fastrtc` + `fastrtc_whisper_cpp` (moonshine) + `kokoro`.

- Arco interno del curso: `Sesion17_Multimodalidad_ANALISIS_COMPLETO.md` (Módulo 6) — el laboratorio de voz de esta sesión es una segunda instancia del Camino 1 (LLM+Tools) ahí documentado, ahora aplicado a audio↔texto con *streaming* en tiempo real;
`Sesion15_LangGraph_MultiAgent_ANALISIS_COMPLETO.md` (Módulo 5) — el patrón *frontline agent* de la diapositiva 8 es la misma topología *Supervisor (as tools)* ahí documentada.
 - `Physical_AI_Agent_Monitoreo_Gatos_Tapo_YOLO_MCP.md` (esta misma carpeta) — documento de diseño propio, candidato directo para la tarea grupal de esta sesión.
-

Documento generado a partir del PDF de la Sesión 18 (Módulo 6, UTEC Posgrado) — 17 diapositivas renderizadas e interpretadas visualmente + código completo del laboratorio adjunto (`agents26_m6s18-main/`) — más investigación propia sobre la guía oficial de OpenAI para agentes de voz.